Running the studio from one screen

Everything `/studio` does, in the order you actually do it. Written against what is deployed, not what is planned. **Where this file and the app disagree, the app is right and this file is stale** — fix it in the same commit that changed the UI.

Getting in

Sign in at **brandmintstudios.in/login** as `admin@brandmintstudios.in`. You land on `/studio`. Anyone who is not that address lands on `/portal` instead, and there is no route from there to the studio screen.

Two shortcuts worth learning on day one:

- `/` or **⌘K** — search every client, project, lead and invoice by name.
- **New** — creates whatever the section you are in creates.

The whole flow, once

Stranger to money in the bank. Nine steps, four screens. Every one of them is explained properly further down — this is only the order they happen in, because the sections below describe each screen well and never put them in a line.

WHERE	WHAT YOU DO
Pipeline → **New**	Log the lead the moment the enquiry arrives
Pipeline → **Log first response**	The moment you reply. Stamped once, never moves
`/quote` → **Print / save as PDF**	Send them the scope and the price
Clients → **New engagement**	They said yes. One submit creates everything
Firebase Console, then **Access**	Give them a login. Two halves, both required
Money → **Invoice**	Bill the first 50%. This is the document with your account on it
Delivery	Raise what you need from them · move a scope line when it moves
Money → **Record payment**	The total received, the day it lands
Money → **Invoice**	At launch, the second 50%

Nothing exists in the database until New engagement. Everything above that row is selling, and a deal that dies there leaves nothing behind to archive. The moment you submit

that form the client, the project, the agreed scope line, both invoices and the milestone schedule all exist at once.

The flow is now on the screen, not just on this page

Open any client and the first thing in the record is **where this client is** — seven stages, the one you are on lit, and the next action written under it.

`Lead → Scope agreed → Agreement signed → Deposit received → In build → Delivered → Settled`

It is worked out from the records themselves: the scope you saved, the agreement you marked, what has actually been paid, and how far the scope lines have moved. **There is nothing there to keep up to date** — no status field to remember, nothing that rots if you do not open it for a fortnight. If it says you are on Deposit received, that is because money arrived, not because somebody ticked a box.

The stage never runs ahead of the facts. An invoice you have raised but not been paid is not a deposit. Some lines delivered is not delivered. Paid in full but not delivered is not settled, and neither is delivered but still owed. A retainer marked *proposed* does not count as an agreement, for the same reason it does not count as revenue.

Agreement signed is a button on the client, and it records rather than signs. Send the document from `/quote` and get it signed by email or Zoho Sign as you do now; the button stamps the date it came back. The date is set by the server, so it cannot be back-dated — and it is what lights the third stage.

Brand kit is on the same record: a logo link, a guidelines link, fonts, and colours as hex. Links, never files — their logo already lives in Figma or Drive and a copy here would be the one that goes stale. It appears in **their portal under Links**, so a client stops emailing you to ask for their own palette. Leave it blank and nothing is shown at all, rather than a row of empty labels.

Two documents, and they are not the same one

The easiest thing here to get wrong, because both of them have a price on.

	QUOTE	INVOICE
Made on	`/quote`	Money → **Invoice**
Sent	**before** they say yes	**after** they say yes
Says	what you get, at what price	what you owe, and where to send it
Payment details	never	yes, once you have entered them

A quotation with your account number on it is asking for money you have not earned yet. So `/quote` has no payment block, is not getting one, and finding no bank details there is the page working correctly. They print on the invoice and nowhere else.

The two also live in different places, which is why deploying this did not make them appear: **the account details are in your database, not in the code.** Go to **Money → Payment details on your invoices**, fill in the four fields, save. This repository is public, so an account

number committed into it would be published permanently to anyone who looks — a form is the fix, not a deploy.

Until account name, number and IFSC are all three present, an invoice prints no payment block whatsoever. That is deliberate: a half-filled one on a financial document a client actually receives is worse than none.

Updates — the screen this is all for

Every live project, one line each, sent together. It is the second entry in the sidebar and it is the one to open on a Tuesday and a Friday.

One box per project. Write what moved, in a sentence, addressed to them. **Save all** writes every one of them at once, and each lands on that client's portal under *Where we are*, dated, above the milestone — because a date tells them when and a sentence tells them what.

Each row then offers **Send on WhatsApp**. That opens WhatsApp with the message already written: the update, how many things are open on their side, and a link to their portal. It does **not** send on its own — you press send, which is the point. A message that went out without being read once is how a wrong number gets a client's update.

It needs a number on the client. Open their record → *Who to message* → a name and a WhatsApp number. A bare ten-digit number is treated as Indian.

The badge on the sidebar counts **only projects nobody has been told about in a week, or ever**. On a week where you have updated everybody it says nothing at all, which is the property that makes it worth looking at.

> **This is the answer to "any update?"** The portal is where a client can go and check. WhatsApp is where they actually read things. Writing it once here does both, and it is the difference between the portal being a thing you maintain and a thing that works for you.

The sixty seconds that matter

Today is the only screen you have to open. Four numbers and a list, all computed from your own data.

This month is signed retainers *plus* what live builds earn per month, shown split and never merged. They behave differently: retainer money comes back next month, build money stops the day it launches.

Two builds running does not double it. Every live build is pooled — total agreed value over total agreed days, times a month of working days — so taking on a third **lengthens the schedule** instead of inflating the month. See `monthlyIncome()` in `bm-app.js` and `tests/income.test.mjs`, which asserts exactly that.

The action list is ordered by what costs most to ignore: an unanswered lead, then clients sitting on your requests, then work awaiting approval, then money. When it is genuinely clear it says so; when the system is *empty* it says that instead, because an empty database and a finished day look identical on most dashboards and mean opposite things.

A proposal is never revenue. A retainer agreed but not marked signed counts as zero everywhere. Today tells you separately how much is sitting unsigned, so the gap is visible without being counted.

A. Taking on a client

One screen, four answers. It replaced seven steps across three pages, which is why it kept getting abandoned halfway.

1. **Clients → New engagement** (the same button is on Delivery).
2. **Name them, then pick the deal** — One-time project · Project then retainer · Retainer only. The form asks only for what that shape needs.
3. **Price and weeks.** One number each. Optionally pick the kind of work, which stamps a milestone schedule for free.
4. **Read the preview.** It writes out exactly what is about to be created — the build, both invoice dates and amounts, and the monthly run rate while it runs. If that sentence is wrong, fix it before submitting rather than after.
5. **Create.** One submit makes the client, the project, the agreed scope line, the 50% now / 50% at launch invoices, and the schedule.

The scope becomes **one** line, not an invented four-phase breakdown — a breakdown nobody agreed to is fiction with a progress bar attached. Refine it on `/quote` when a real one exists.

No path through this form marks a retainer signed unless you tick the box.

B. Giving them a login

The one job that happens in two places, and the one most likely to be left half-done.

1. **Firebase Console → Authentication → Users → Add user.** Email is `theirname@brandmintstudios.in`. **Set the password there.** Copy the UID.
2. **Studio → Access.** Username, display name, organisation, paste the UID. The form shows the exact email they will sign in with before you save.
3. **Check it says "Saved."** They appear under *Existing logins* marked `linked`.

> **Do both halves or neither works.** The Firebase account and the `users` document are separate writes. A client with the account but no `users` document signs in successfully and then **reads nothing at all** — not even their own company name, because the rules resolve every read through that document. A blank portal almost always means step 2 was skipped.

Never collect a client credential. No screen here accepts one and you should not take one by email either. When a client needs to give you access to something of theirs, ask them to add `hello@brandmintstudios.in` as a user on their own account.

The three ways you sell

DEAL	BUILD	RETAINER	WHAT DELIVERY SHOWS
One-time	yes, 50/50	none, by agreement	% delivered, from the scope
Then retainer	yes, 50/50	starts at launch	% delivered, then ongoing
Retainer only	no	the whole engagement	**Ongoing** — no end date, by design

A retainer engagement is never nagged about missing milestones. It does not have any and is not supposed to.

What each section is for

SECTION	FOR
Today	The month, and what needs you. Open this one.
Updates	Every live project, a line each, sent to their portal and their WhatsApp in one pass.
Pipeline	Leads as a board. **Log first response the moment you reply** — stamped once, never moves, and the most valuable number in the business.
Clients	One row each → **where they are in the flow**, agreement, brand kit, retainer, projects, invoices, login, and which onboarding step is next. Mark a retainer signed here.
Delivery	One row per project → move scope lines `agreed → building → delivered → accepted` with a click, raise what you need from the client, put work up for approval.
Money	Every invoice ever raised. *Record payment* takes the **total received**, so a part payment is a real number.
Access	Client logins, and who has one.
Activity	The audit log. No longer in the sidebar — reach it from the command palette (`/` or ⌘K). Append-only; nobody can edit or delete an entry, **including you**.

The daily habit

Three things, none longer than a minute, and between them every number on the screen stays true.

Raise what you need from the client. Delivery → open the project → add each item under *What we need from the client*. This is the whole reason the portal exists: it shows them, dated, so you are not the one chasing. A project with nothing raised has that switched off, and Today will tell you so.

Move a scope line when it moves. One click. The percentage, the client's portal and the month all follow without anything being typed twice. A line counts as money only at **accepted** — delivered means handed over, not confirmed.

Record money the day it lands. Money → *Record payment* → the total received. For anything paid online use **Sync payments** instead: it asks Razorpay what it was actually paid rather than taking your word for it.

Still outstanding

- **Revoke the old service-account key `090ec957…`.** It was pasted into a chat once, which makes it spent whatever happened to the file. Firebase Console → Project settings → Service accounts. Nothing in the app needs it.
- **Publish `firestore.rules`. This is now the blocking item.** Firestore → Rules → paste the whole file → Publish. It carries the five-role permission matrix — CEO, Partner, Collaborator, Finance, Client — and until it is published none of those roles exists in the database, only in the file. The Activity log also records nothing until then, and says so plainly.

Publish before granting anyone a login. The rules are stricter than what is live, so everything already deployed keeps working against them; the reverse is not true. A collaborator given an account against the old ruleset is a collaborator with no restrictions at all.

Afterwards, confirm what is actually live rather than trusting the dialog: Console → Firestore → Rules shows the published source, and it should match the file character for character.

- **Prove tenancy on live data.** Sign in as a real client and open `/tenancy-check`. It should say **Isolated**. It is tested six ways against the emulator and has never been run against the live database — §9 item 1.

Brand Mint Studios · generated from `docs/WALKTHROUGH.md`. Regenerate with `node tools/make-manual.mjs` whenever the admin screen changes.